

Domain-Driven Design (DDD) in ASP.NET Core Web API

In this session, we will explore what DDD is, why it matters, its core layers, and implement a real-time E-Commerce System using DDD principles with ASP.NET Core Web API.

Introduction to Domain-Driven Design (DDD)

Modern business applications often deal with complex business rules and processes that require a deep understanding of the domain, the problem space for which the software is being built. Domain-Driven Design (DDD) is an architectural approach and methodology focused on modelling software to reflect the core business domain and its logic closely.

Domain-Driven Design (DDD) is an approach to software development introduced by **Eric Evans** in his book *Domain-Driven Design: Tackling Complexity in the Heart of Software*. The primary goal of DDD is to create a **Shared Understanding** of the **Business Domain** between Developers and **Domain Experts**, and to use that understanding to build a software model that truly represents real-world business processes.

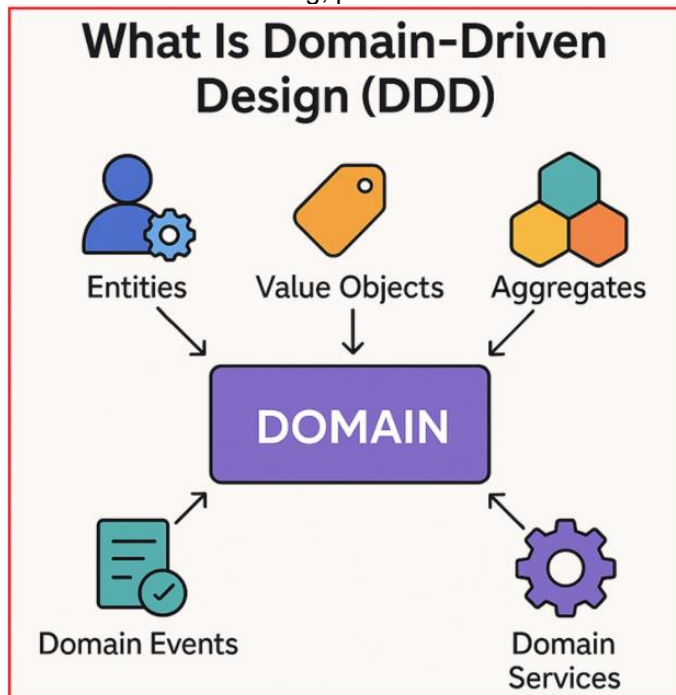
What Is Domain-Driven Design (DDD)?

Domain-Driven Design (DDD) Architecture is a method of structuring software so that the domain, meaning the business problem or area you are solving, is placed at the center of your design. In DDD, we first build a conceptual model of our business, capturing all the important objects, rules, and behaviours.

This Domain model consists of several building blocks:

- **Entities:** Domain objects that have a distinct identity, remaining the same even as their attributes change.
- **Value Objects:** Immutable objects defined solely by their properties and values, not by identity.
- **Aggregates:** A cluster of related entities and value objects treated as a single unit for data changes and consistency.
- **Domain Events:** Records of something important that has happened within the domain, used to trigger side effects or communication.
- **Domain Services:** Operations or business logic that don't naturally fit within entities or value objects but are essential to the domain.

For a better understanding, please have a look at the following image.

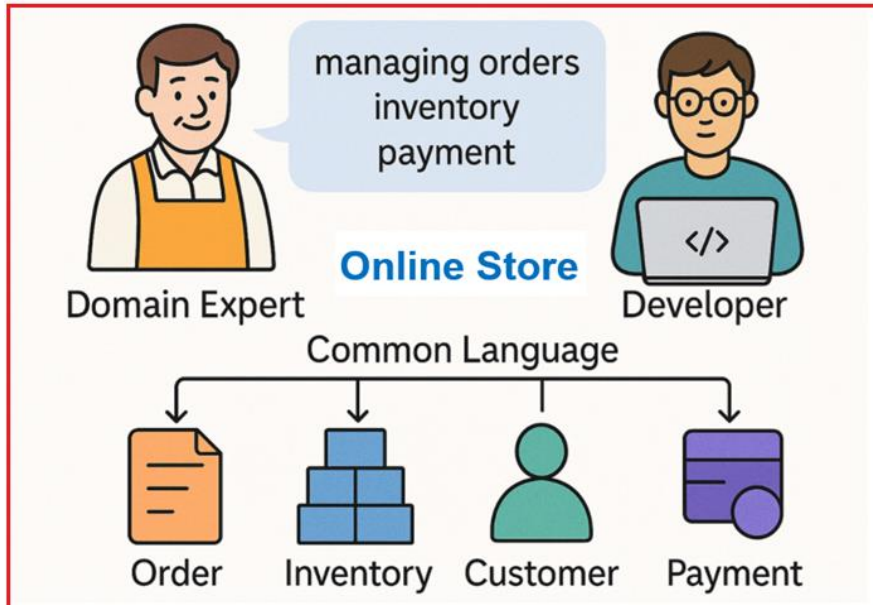


By focusing on the domain, DDD helps ensure that your code accurately reflects the way your business actually works, making it easier to understand, maintain, and evolve.

Layman Examples to understand Domain-Driven Design (DDD)

Let us understand the need for Domain Driven Design with a few layman examples.

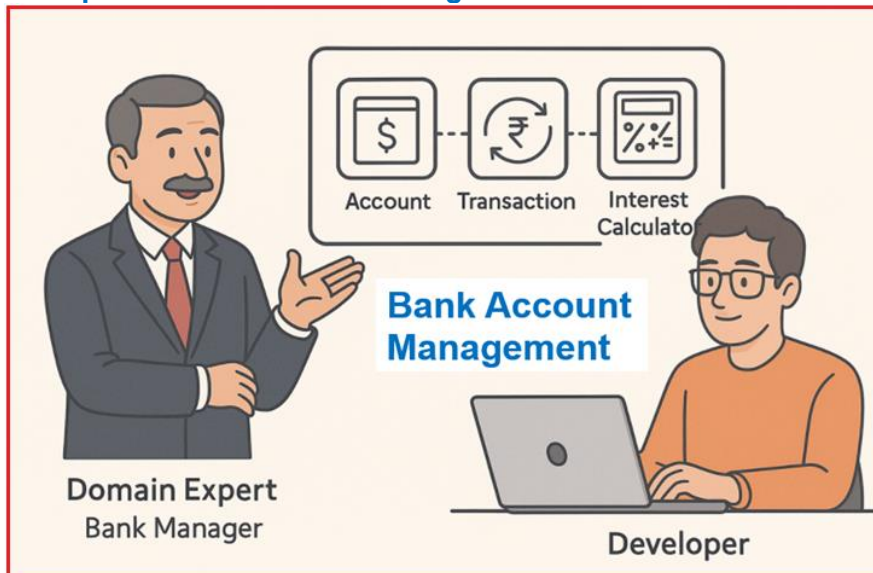
Example 1: Online Store



Here:

- **Domain Expert (Bookstore Owner):** Talks about managing orders, keeping track of inventory, customer information, and handling payments.
- **Developer:** Creates objects like Order, Inventory, Customer, and Payment in code, each encapsulating real-world business rules, for instance, reducing stock when an order is placed or validating payment details.
- **Common Language:** Both domain expert and developers use the same terms and concepts like "order," "inventory," and "payment" to avoid miscommunication and ensure clarity.

Example 2: Bank Account Management



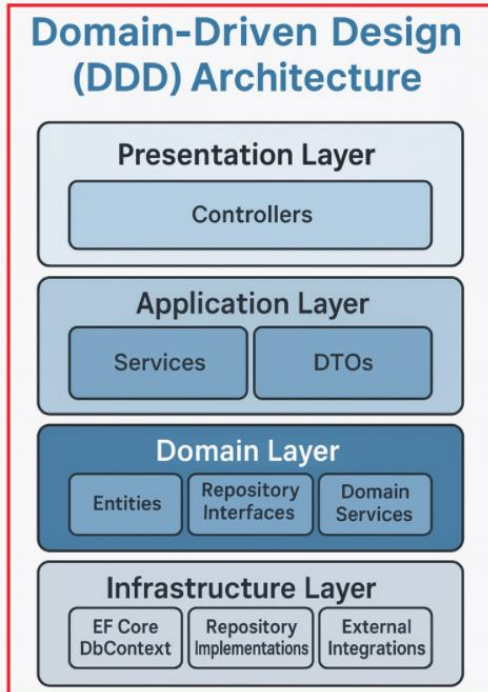
Here:

- **Domain Expert (Bank Manager):** Discusses accounts, transactions, overdraft limits, and how interest is calculated.
- **Developer:** Models these as Account, Transaction, and InterestCalculator objects. All the rules (like not allowing overdrafts beyond ₹10,000 or calculating monthly interest) live inside these objects, not scattered around in controllers or database scripts.

- **Common Language:** Everyone calls it an “Account” and a “Transaction,” so requirements and code are always in sync and mistakes are minimized.

Domain-Driven Design (DDD) Architecture

Domain-Driven Design (DDD) structures an application into distinct layers. This separation of concerns leads to cleaner, more maintainable code and helps ensure that business rules remain at the heart of the system. For a better understanding, please refer to the following diagram:



The following is a detailed explanation of each layer and its core components:

Domain Layer (Entities, Repository Interfaces, Domain Services)

The Domain Layer is the core of DDD and contains the pure business logic and domain model. It models real-world business concepts, rules, and behaviors, independent of technical concerns. The following are the Key Components:

- **Entities:** Business objects with a unique identity that persists over time (identity never changes). They contain state and business behavior (e.g., Product, Order, Customer).
- **Repository Interfaces:** Define contracts for accessing and storing entities. They define methods for operations like finding orders or saving products, with implementations provided in the Infrastructure Layer.
- **Domain Services:** Services that encapsulate domain logic which doesn't naturally fit within a single entity (e.g., complex business calculations or operations spanning multiple entities).

Example: The Order entity enforces rules like “an order must have at least one item.” The `IOrderRepository` interface defines how orders are persisted, but the actual database code is elsewhere.

Infrastructure Layer (EF Core DbContext, Repository Implementations, External Integrations)

The Infrastructure Layer contains all technical implementation details related to external systems and frameworks (databases, message brokers, files, third-party services). It keeps infrastructure concerns isolated so the Domain Layer remains clean and independent. The following are the Key Components:

- **EF Core DbContext:** Acts as a bridge between domain models and the database. Manages querying, saving, and updating entities (e.g., Product, Order) within the database.
- **Repository Implementations:** Actual code for storing and retrieving entities, as defined by the repository interfaces in the Domain Layer (e.g., `OrderRepository` using EF Core).

- **External Integrations:** Services communicating outside the core system, such as email services, payment gateways, messaging queues, or third-party APIs. Placed here to avoid polluting domain and application logic.

Example: OrderRepository implements IOrderRepository to interact with SQL Server using EF Core. The EmailService integration sends confirmation emails when an order is placed.

Application Layer (DTOs and Services)

The Application Layer acts as a coordinator that connects the Presentation Layer with the Domain Layer. It manages the flow of data, business workflows, and enforces application-specific policies. This layer does **not** contain core business rules; those reside in the Domain Layer. The following are the Key Components:

DTOs (Data Transfer Objects):

- Simple objects used to carry data between the Presentation Layer and the Application Layer.
- They exclude business behavior and contain only the necessary data for interaction.
- DTOs help decouple the domain model from external data formats, improving security and flexibility.
- Examples include OrderCreatedDTO for incoming order data and OrderDTO for responses.

Services (Application Services):

- Represent the system's use cases and business workflows, such as placing orders, processing payments, or managing product lifecycles.
- Handle tasks like transaction management, authorization, and calling multiple domain entities or services to complete a use case.
- Delegate core domain logic to domain entities and domain services to maintain separation of concerns.

Example: An OrderService method PlaceOrderAsync() takes validated input (DTOs) from the Presentation Layer, calls domain methods to create and validate an order, and manages the overall order processing flow.

Presentation Layer (Controllers)

The Presentation Layer is responsible for handling all incoming requests from users or external systems and managing how the application interacts with its clients. It serves as the interface between the outside world and the internal system. The following are the Key Components of the Presentation Layer:

Controllers:

- Act as the entry points for external clients such as browsers, mobile apps, or other services.
- Handle receiving HTTP requests, performing input validation (checking format and correctness), and returning appropriate HTTP responses (e.g., JSON data, status codes).
- Designed to be thin and lightweight, delegating business workflows and logic to the Application Layer.

Example: An OrderController receives a POST request, validates it, and calls the Application Layer's OrderService to process the order. The result is returned as a response.

Real-time E-Commerce Application using Domain-Driven Design (DDD) Principles:

Domain-Driven Design puts the business domain, the rules and processes that define your business, at the heart of your application. Layering separates concerns, making your code more understandable, testable, and adaptable as the business grows or changes.

Let's start building a Real-time E-Commerce Application using Domain-Driven Design (DDD) principles, step-by-step, explaining each layer, domain element, and their responsibilities.

Solution Structure and Layer Setup

We organize the solution into four distinct projects (layers), each with a clear responsibility to maintain separation of concerns and follow DDD principles:

ECommerce.Domain

- This will be a class library project and it will contain the core business logic and domain model. This includes Entities, Value Objects, Aggregates, Domain Services, and Repository Interfaces.
- This layer is independent and has no references to other layers, keeping the domain pure.

ECommerce.Infrastructure

- This will be a class library project and it will implement data access and external services, like email or payment gateways. It contains EF Core DbContext and concrete Repository Implementations that persist domain entities.
- This layer depends on the Domain layer to implement repository interfaces.

ECommerce.Application

- This will be a class library project and it will coordinate the workflows between domain entities and infrastructure. It contains Application Services, DTOs for data transfer, and service interfaces and its implementations.
- This layer depends on Domain (for entities and repository interfaces) and Infrastructure (for repository implementations) layers.

ECommerce.API

- This will be an ASP.NET Core Web API Project and it exposes the API endpoints as controllers, handles HTTP requests/responses, and communicates only with the Application layer using DTOs. The client will interact with this layer.
- This layer depends only on the Application layer, never on Domain or Infrastructure.

Why Clear Layering or Benefits of DDD?

- **Maintainability:** Each layer focuses on a single concern, making the code easier to understand and maintain.
- **Testability:** Layers can be tested independently (e.g., domain logic without database).
- **Flexibility:** You can replace UI or infrastructure details without affecting the core business logic.
- **Collaboration:** Domain experts and developers can focus on the domain model without worrying about technical details.

Create the Empty Solution and Add Projects

First, create a Blank Project Solution named **ECommerceDDD**. Once you created the Blank Project Solution, then add 3 classes library project and 1 ASP.NET Core Web API project to the Blank solution.

Class Library Projects:

Please provide the class library project names as follows:

- ECommerce.Domain
- ECommerce.Infrastructure
- ECommerce.Application

ASP.NET Core Web API Project:

Please provide the ASP.NET Core Web API project name as follows:

- ECommerce.API

Note: To make the project clean and start everything from scratch, please remove the default **Class1.cs** class files from the Class Library Projects and the default **WeatherForecastController.cs**, **WeatherForecast.cs** files from the ASP.NET Core Web API Project. Also, please mark the ECommerce.API as the startup project.

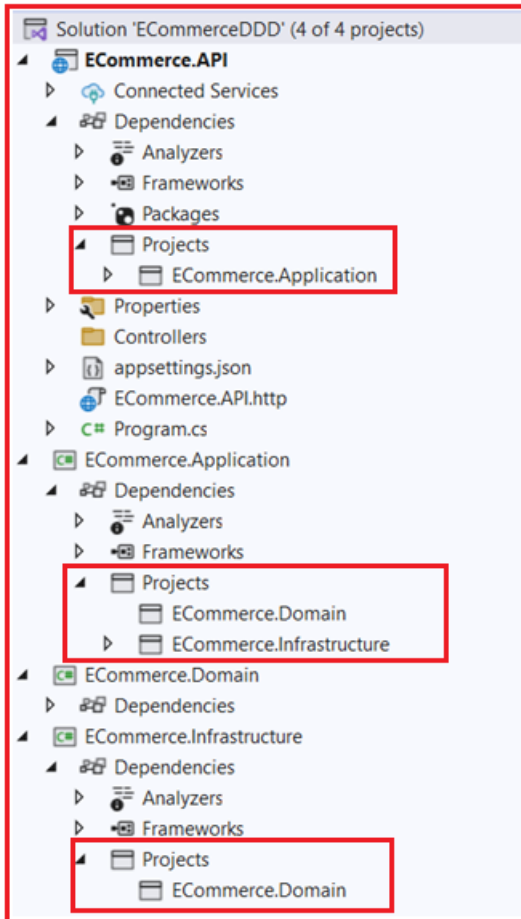
Add Project References

Now, we need to add Project References of the dependency Projects as follows:

- **ECommerce.Infrastructure** → **ECommerce.Domain**: To implement repository interfaces and work with domain entities.
- **ECommerce.Application** → **ECommerce.Domain**, **ECommerce.Infrastructure**: To use domain models and infrastructure implementations (e.g., repositories).

- **ECommerce.API** → **ECommerce.Application**, **ECommerce.Domain**, **ECommerce.Infrastructure**: To Register Repository Service, Application Services, DbContext and call application services and use DTOs.

No references from Domain to Infrastructure or Application to maintain domain purity. With the above project reference, your solution with projects should look as shown in the image below:



In summary:

- Infrastructure depends on Domain.
- Application depends on Domain and Infrastructure.
- API depends on Application.
- Domain has no dependencies on any other project.

Domain Layer: Defining Core Business Model

Let us first implement the Domain Layer which is core of DDD. **This layer defines the core business entities and logic.** Some of the Entities need EF Core Attributes. So, please add the following Package using NuGet Package Managers for Solution or by executing the following commands in Visual Studio Package Manager Console. While executing these commands, please select the **ECommerce.Domain** class library project.

Install-Package Microsoft.EntityFrameworkCore.SqlServer

Define Entities

So, first create a folder named **Entities**, within the root directory of **ECommerce.Domain** class library project where we will create all the above entities:

Product (Entity)

Create a class file named **Product.cs** within the **Entities** folder of ECommerce.Domain class library project, then copy and paste the following code. It represents a product in the store (with Id, Name, Price, Stock, etc.) and has methods to change price or reduce stock.

```

using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace ECommerce.Domain.Entities
{
    public class Product
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; private set; }

        [Required, MaxLength(200)]
        public string Name { get; private set; } = null!;

        [Column(TypeName = "decimal(12,2)")]
        public decimal Price { get; private set; }
        [MaxLength(1000)]
        public string? Description { get; private set; }
        public int StockQuantity { get; private set; }

        // Navigation property for OrderItems referencing this product (optional)
        public ICollection<OrderItem> OrderItems { get; private set; } = new List<OrderItem>();

        private Product() { } // EF Core needs parameterless constructor

        public Product(string name, decimal price, int stockQuantity, string? description)
        {
            Name = name;
            Price = price;
            StockQuantity = stockQuantity;
            Description = description;
        }

        public void ChangePrice(decimal newPrice)
        {
            if (newPrice <= 0)
                throw new ArgumentException("Price must be positive");
            Price = newPrice;
        }

        public void ReduceStock(int quantity)
        {
            if (quantity > StockQuantity)
                throw new InvalidOperationException("Insufficient stock");
            StockQuantity -= quantity;
        }
    }
}

```

Customer (Entity)

Create a class file named **Customer.cs** within the **Entities** folder of ECommerce.Domain class library project, then copy and paste the following code. It represents a user who can place orders (with Id, Name, Email, Address) and also has navigation to orders placed by the customer.

```

using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;
using Microsoft.EntityFrameworkCore;

namespace ECommerce.Domain.Entities
{

```

```

[Index(nameof(Email), Name = "Index_RegistrationNumber", IsUnique = true)]
public class Customer
{
    [Key]
    [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
    public int Id { get; private set; }

    [Required, MaxLength(100)]
    public string FirstName { get; private set; } = null!;

    [MaxLength(100)]
    public string? LastName { get; private set; }

    [Required, EmailAddress]
    public string Email { get; private set; } = null!;

    public Address Address { get; private set; } = null!;

    // Navigation property for Orders of this Customer
    public ICollection<Order> Orders { get; private set; } = new List<Order>();

    private Customer() { }

    public Customer(string firstName, string lastName, string email, Address address)
    {
        FirstName = firstName;
        LastName = lastName;
        Email = email;
        Address = address;
    }
}

```

Address (Value Object)

Create a class file named **Address.cs** within the **Entities** folder of ECommerce.Domain class library project, then copy and paste the following code. It represents a customer or shipping address with no identity, only value. It captures street, city, state, postal code, and country. Used by Customer and Order as an owned value object (embedded in table, not separate).

```

using System.ComponentModel.DataAnnotations;
using Microsoft.EntityFrameworkCore;

namespace ECommerce.Domain.Entities
{
    [Owned]
    public sealed class Address
    {
        [Required, MaxLength(250)]
        public string Street { get; private set; } = null!;

        [Required, MaxLength(100)]
        public string City { get; private set; } = null!;

        [Required, MaxLength(100)]
        public string State { get; private set; } = null!;

        [Required, MaxLength(20)]
        public string PostalCode { get; private set; } = null!;

        [Required, MaxLength(100)]

```

```

    public string Country { get; private set; } = null!;

    private Address() { }

    public Address(string street, string city, string state, string postalCode, string country)
    {
        Street = street;
        City = city;
        State = state;
        PostalCode = postalCode;
        Country = country;
    }
}
}

```

Order (Aggregate Root)

Create a class file named **Order.cs** within the **Entities** folder of ECommerce.Domain class library project, then copy and paste the following code. It represents a customer's order (with Id, Customer, Date, Total, Shipping Address) and is the Aggregate Root for Order-related operations. It ensures business rules (e.g., stock availability, minimum order quantity) by controlling additions and modifications.

```

using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;

namespace ECommerce.Domain.Entities
{
    public class Order
    {
        [Key]
        [DatabaseGenerated(DatabaseGeneratedOption.Identity)]
        public int Id { get; private set; }

        [Required]
        public int CustomerId { get; private set; }

        public DateTime OrderDate { get; private set; }

        public Address ShippingAddress { get; private set; } = null!;

        [Column(TypeName = "decimal(12,2)")]
        public decimal TotalAmount { get; private set; }

        // Navigation to Customer
        public Customer Customer { get; private set; } = null!;

        // Navigation collection for OrderItems
        public ICollection<OrderItem> Items { get; private set; } = new List<OrderItem>();

        private Order() { }

        public Order(int customerId, Address shippingAddress)
        {
            CustomerId = customerId;
            ShippingAddress = shippingAddress;
            OrderDate = DateTime.UtcNow;
        }

        public void AddItem(Product product, int quantity)
        {

```

```

        if (quantity <= 0)
            throw new ArgumentException("Quantity must be greater than zero");

        if (product.StockQuantity < quantity)
            throw new InvalidOperationException("Not enough stock");

        var existingItem = Items.FirstOrDefault(i => i.ProductId == product.Id);
        if (existingItem != null)
        {
            existingItem.IncreaseQuantity(quantity);
        }
        else
        {
            Items.Add(new OrderItem(product.Id, product.Name, product.Price, quantity, this));
        }

        product.ReduceStock(quantity);
        CalculateTotalAmount();
    }

    private void CalculateTotalAmount()
    {
        TotalAmount = Items.Sum(i => i.UnitPrice * i.Quantity);
    }
}

```

OrderItem (Entity inside Order aggregate)

Create a class file named **OrderItem.cs** within the **Entities** folder of ECommerce.Domain class library project, then copy and paste the following code. It represents a line item in an order (Product, Quantity, Price).

```

using System.ComponentModel.DataAnnotations.Schema;
using System.ComponentModel.DataAnnotations;

namespace ECommerce.Domain.Entities
{
    public class OrderItem
    {
        public int OrderId { get; private set; }
        public int ProductId { get; private set; }

        [MaxLength(200)]
        public string? ProductName { get; private set; }

        [Column(TypeName = "decimal(12,2)")]
        public decimal UnitPrice { get; private set; }

        public int Quantity { get; private set; }
        public Order Order { get; private set; } = null!;

        private OrderItem() { } // EF Core

        public OrderItem(int productId, string productName, decimal unitPrice, int quantity, Order order)
        {
            ProductId = productId;
            ProductName = productName;
            UnitPrice = unitPrice;
            Quantity = quantity;
        }
    }
}

```

```

        Order = order ?? throw new ArgumentNullException(nameof(order));
        OrderId = order.Id;
    }

    public void IncreaseQuantity(int quantity)
    {
        Quantity += quantity;
    }
}
}

```

Defining Repository Interfaces

Repository Interfaces define contracts for data access to keep domain independent of infrastructure. Create a folder named **Repositories** within the root directory of **ECommerce.Domain** class library project where we will create all our Repository Interfaces:

IProductRepository

Create a class file named **IProductRepository.cs** within the **Repositories** folder of ECommerce.Domain class library project, then copy and paste the following code. It defines CRUD methods for Product.

```

using ECommerce.Domain.Entities;

namespace ECommerce.Domain.Repositories
{
    public interface IProductRepository
    {
        Task<Product?> GetByIdAsync(int id);
        Task<IEnumerable<Product>> GetAllAsync();
        Task<Product> AddAsync(Product product);
        Task UpdateAsync(Product product);
    }
}

```

IOrderRepository

Create a class file named **IOrderRepository.cs** within the **Repositories** folder of ECommerce.Domain class library project, then copy and paste the following code. It defines Methods to add orders and retrieve by Id.

```

using ECommerce.Domain.Entities;

namespace ECommerce.Domain.Repositories
{
    public interface IOrderRepository
    {
        Task<Order?> GetByIdAsync(int id);
        Task AddAsync(Order order);
    }
}

```

ICustomerRepository

Create a class file named **ICustomerRepository.cs** within the **Repositories** folder of ECommerce.Domain class library project, then copy and paste the following code. It defines Methods to get or add Customer.

```

using ECommerce.Domain.Entities;

namespace ECommerce.Domain.Repositories
{
    public interface ICustomerRepository
    {
        Task<Customer?> GetByIdAsync(int id);
    }
}

```

```

        Task AddAsync(Customer customer);
    }
}

```

Defining Domain Services:

Encapsulate business logic that doesn't belong in a single entity/value object. First, create a folder named **Services** within the root directory of **ECommerce.Domain** class library project where we will create all our domain services.

OrderDomainService:

Create a class file named **OrderDomainService.cs** within the **Services** folder of **ECommerce.Domain** class library project, then copy and paste the following code. Contains domain logic that involves multiple entities, e.g., checking if an order can be placed based on customer existence and item availability.

```

using ECommerce.Domain.Entities;
namespace ECommerce.Domain.Services
{
    public class OrderDomainService
    {
        public bool CanPlaceOrder(Customer customer, List<OrderItem> items)
        {
            // Business logic: e.g., customer must exist, items should be in stock, etc.
            return customer != null && items != null && items.Count > 0;
        }
    }
}

```

Infrastructure Layer — EF Core Implementation

Let us implement the Infrastructure Layer where we will create the DbContext class and the Repository Interface implementations. As we will be writing the core data access logic within this project which will use EF Core Code First Approach with SQL Server database, so first we need to install the required packages.

Adding Packages:

So, please add the following Package using NuGet Package Managers for Solution or by executing the following commands in Visual Studio Package Manager Console. While executing these commands, please select the **ECommerce.Infrastructure** class library project.

```

Install-Package Microsoft.EntityFrameworkCore.SqlServer
Install-Package Microsoft.EntityFrameworkCore.Tools

```

Next, create two folders named **Persistence**, and **Repositories** at the project root directory of the **ECommerce.Infrastructure** project.

ECommerceDbContext

Create a class file named **ECommerceDbContext.cs** within the **Persistence** folder of **ECommerce.Infrastructure** class library project, then copy and paste the following code. EF Core DbContext defining DbSet for Products, Customers, Orders, and OrderItems. Configures entity relationships, composite keys, and seeds initial data.

```

using Microsoft.EntityFrameworkCore;
using ECommerce.Domain.Entities;

namespace ECommerce.Infrastructure.Persistence
{
    public class ECommerceDbContext : DbContext
    {
        public ECommerceDbContext(DbContextOptions<ECommerceDbContext> options) :
            base(options) { }
    }
}

```

```

public DbSet<Product> Products { get; set; } = null!;
public DbSet<Customer> Customers { get; set; } = null!;
public DbSet<Order> Orders { get; set; } = null!;
public DbSet<OrderItem> OrderItems { get; set; } = null!;

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Address as Value Object for Customer and Order
    modelBuilder.Entity<Customer>().OwnsOne(c => c.Address);
    modelBuilder.Entity<Order>().OwnsOne(o => o.ShippingAddress);

    //Composite Primary Key
    modelBuilder.Entity<OrderItem>()
        .HasKey(oi => new { oi.OrderId, oi.ProductId });

    //Specifying the Custom column names for Owned Entity properties
    //By Default: ShippingAddress_Street, ShippingAddress_City, etc.
    //Changing to: Street, City, etc.
    modelBuilder.Entity<Order>(builder =>
    {
        builder.OwnsOne(o => o.ShippingAddress, sa =>
        {
            sa.Property(p => p.Street).HasColumnName("Street");
            sa.Property(p => p.City).HasColumnName("City");
            sa.Property(p => p.State).HasColumnName("State");
            sa.Property(p => p.PostalCode).HasColumnName("PostalCode");
            sa.Property(p => p.Country).HasColumnName("Country");
        });
    });

    modelBuilder.Entity<Customer>(builder =>
    {
        builder.OwnsOne(c => c.Address, a =>
        {
            a.Property(p => p.Street).HasColumnName("Street");
            a.Property(p => p.City).HasColumnName("City");
            a.Property(p => p.State).HasColumnName("State");
            a.Property(p => p.PostalCode).HasColumnName("PostalCode");
            a.Property(p => p.Country).HasColumnName("Country");
        });
    });

    // Seed Customers (owner entity)
    modelBuilder.Entity<Customer>().HasData(
        new { Id = 1, FirstName = "Pranaya", LastName = "Rout", Email =
"pranaya@example.com" },
        new { Id = 2, FirstName = "Priyanka", LastName = "Sharma", Email =
"priyanka@example.com" }
    );

    // Seed Addresses (owned entity) separately, link via CustomerId
    modelBuilder.Entity<Customer>().OwnsOne(c => c.Address).HasData(
        new
        {
            CustomerId = 1,
            Street = "123 Main Road",
            City = "Bhubaneswar",
            State = "Odisha",
            PostalCode = "751024",
            Country = "India"
        }
    );
}

```

```

    },
    new
    {
        CustomerId = 2,
        Street = "45 Park Avenue",
        City = "Delhi",
        State = "Delhi",
        PostalCode = "110001",
        Country = "India"
    }
);

// Seed Products
modelBuilder.Entity<Product>().HasData(
    new { Id = 1, Name = "Apple iPhone 15", Price = 70000m, StockQuantity = 50,
    Description = "Latest iPhone with A17 chip and improved camera." },
    new { Id = 2, Name = "Samsung Galaxy S24", Price = 65000m, StockQuantity = 70,
    Description = "Flagship Samsung with dynamic AMOLED display." },
    new { Id = 3, Name = "OnePlus Nord 4", Price = 32000m, StockQuantity = 80,
    Description = "Affordable 5G smartphone with smooth performance." }
);
}
}
}

```

ProductRepository

Create a class file named **ProductRepository.cs** within the **Repositories** folder of ECommerce.Infrastructure class library project, then copy and paste the following code. EF Core implementation of IProductRepository.

```

using ECommerce.Domain.Entities;
using ECommerce.Domain.Repositories;
using ECommerce.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace ECommerce.Infrastructure.Repositories
{
    public class ProductRepository : IProductRepository
    {
        private readonly ECommerceDbContext _dbContext;

        public ProductRepository(ECommerceDbContext dbContext)
        {
            _dbContext = dbContext;
        }

        public async Task<Product> AddAsync(Product product)
        {
            await _dbContext.Products.AddAsync(product);
            await _dbContext.SaveChangesAsync();
            return product;
        }

        public async Task<IEnumerable<Product>> GetAllAsync()
        {
            return await _dbContext.Products.AsNoTracking().ToListAsync();
        }

        public async Task<Product?> GetByldAsync(int id)
        {

```

```

        return await _dbContext.Products.FindAsync(id);
    }

    public async Task UpdateAsync(Product product)
    {
        _dbContext.Products.Update(product);
        await _dbContext.SaveChangesAsync();
    }
}

```

OrderRepository

Create a class file named **OrderRepository.cs** within the **Repositories** folder of ECommerce.Infrastructure class library project, then copy and paste the following code. EF Core implementation of ICustomerRepository.

```

using Microsoft.EntityFrameworkCore;
using ECommerce.Domain.Entities;
using ECommerce.Domain.Repositories;
using ECommerce.Infrastructure.Persistence;

namespace ECommerce.Infrastructure.Repositories
{
    public class OrderRepository : IOrderRepository
    {
        private readonly ECommerceDbContext _dbContext;

        public OrderRepository(ECommerceDbContext dbContext)
        {
            _dbContext = dbContext;
        }

        public async Task AddAsync(Order order)
        {
            await _dbContext.Orders.AddAsync(order);
            await _dbContext.SaveChangesAsync();
        }

        public async Task<Order?> GetByldAsync(int id)
        {
            return await _dbContext.Orders
                .Include(o => o.Items) //Eager Loading
                .AsNoTracking()
                .FirstOrDefaultAsync(o => o.Id == id);
        }
    }
}

```

CustomerRepository

Create a class file named **CustomerRepository.cs** within the **Repositories** folder of ECommerce.Infrastructure class library project, then copy and paste the following code. EF Core implementation of IOrderRepository.

```

using ECommerce.Domain.Entities;
using ECommerce.Domain.Repositories;
using ECommerce.Infrastructure.Persistence;
using Microsoft.EntityFrameworkCore;

namespace ECommerce.Infrastructure.Repositories
{
    public class CustomerRepository : ICustomerRepository

```

```

{
    private readonly ECommerceDbContext _dbContext;

    public CustomerRepository(ECommerceDbContext dbContext)
    {
        _dbContext = dbContext;
    }

    public async Task<Customer?> GetByIdAsync(int id)
    {
        return await _dbContext.Customers
            .Include(c => c.Address) // Include owned Address value object
            .FirstOrDefaultAsync(c => c.Id == id);
    }

    public async Task AddAsync(Customer customer)
    {
        await _dbContext.Customers.AddAsync(customer);
        await _dbContext.SaveChangesAsync();
    }
}
}

```

Application Layer — Services and DTOs

Let us implement the Application Layer where we will create the DTOs, the Service Interface and implementation and AutoMapper Mapping classes. First create three folders named **DTOs**, **Mappings**, and **Services** at the project root directory of the **ECommerce.Application** project.

In this project, we will use AutoMapper, so please install AutoMapper package by executing the below command in Package Manager Console:

Install-Package AutoMapper.Extensions.Microsoft.DependencyInjection

Creating DTOs:

Data Transfer Objects (DTOs) simplify data exchanged with external layers (API/UI). They contain only necessary data and validation attributes. Examples: ProductDTO, CreateProductDTO, OrderDTO, AddressDTO.

ProductDTO

Create a class file named **ProductDTO.cs** within the DTOs folder of ECommerce.Application class library project, then copy and paste the following code. This DTO is used for returning product details.

```

namespace ECommerce.Application.DTOs
{
    public class ProductDTO
    {
        public int Id { get; set; }
        public string Name { get; set; } = null!;
        public string? Description { get; set; }
        public decimal Price { get; set; }
        public int StockQuantity { get; set; }
    }
}

```

CreateProductDTO

Create a class file named **CreateProductDTO.cs** within the DTOs folder of ECommerce.Application class library project, then copy and paste the following code. This DTO is used for incoming product creation requests.

```

using System.ComponentModel.DataAnnotations;

```

```

namespace ECommerce.Application.DTOs
{
    public class CreateProductDTO
    {
        [Required, MaxLength(200)]
        public string Name { get; set; } = null!;

        public string? Description { get; set; }

        [Range(0.01, double.MaxValue)]
        public decimal Price { get; set; }

        [Range(0, int.MaxValue)]
        public int StockQuantity { get; set; }
    }
}

```

Application Services

Implement business workflows that use repositories and domain services, map domain entities to DTOs (and vice versa) using AutoMapper, and enforce application-specific rules.

IProductService Interface

Create a class file named **IProductService.cs** within the Services folder of ECommerce.Applicaion class library project, then copy and paste the following code. Expose methods to get all products, get by Id, and create new product.

```
using ECommerce.Application.DTOs;
```

```

namespace ECommerce.Application.Services
{
    public interface IProductService
    {
        Task<IEnumerable<ProductDTO>> GetAllProductsAsync();
        Task<ProductDTO?> GetProductByIdAsync(int id);
        Task<ProductDTO> AddProductAsync(CreateProductDTO productDto);
    }
}

```

ProductService Implementation

Create a class file named **ProductService.cs** within the **Services** folder of ECommerce.Applicaion class library project, then copy and paste the following code. It uses repository for data access and AutoMapper for mapping domain objects to DTOs.

```

using AutoMapper;
using ECommerce.Application.DTOs;
using ECommerce.Domain.Entities;
using ECommerce.Domain.Repositories;

namespace ECommerce.Application.Services
{
    public class ProductService : IProductService
    {
        private readonly IProductRepository _productRepository;
        private readonly IMapper _mapper;

        public ProductService(IProductRepository productRepository, IMapper mapper)
        {
            _productRepository = productRepository;
            _mapper = mapper;
        }
    }
}

```

```

        public async Task<ProductDTO> AddProductAsync(CreateProductDTO productDto)
        {
            var product = new Product(productDto.Name, productDto.Price,
productDto.StockQuantity, productDto.Description);
            var createdProduct = await _productRepository.AddAsync(product);
            return _mapper.Map<ProductDTO>(createdProduct);
        }

        public async Task<IEnumerable<ProductDTO>> GetAllProductsAsync()
        {
            var products = await _productRepository.GetAllAsync();
            return _mapper.Map<IEnumerable<ProductDTO>>(products);
        }

        public async Task<ProductDTO?> GetProductByldAsync(int id)
        {
            var product = await _productRepository.GetByldAsync(id);
            return product == null ? null : _mapper.Map<ProductDTO>(product);
        }
    }
}

```

OrderDTO

Create a class file named **OrderDTO.cs** within the DTOs folder of ECommerce.Applicaion class library project, then copy and paste the following code. This DTO is used for returning order details.

```

namespace ECommerce.Application.DTOs
{
    public class OrderDTO
    {
        public int Id { get; set; }
        public int CustomerId { get; set; }
        public decimal TotalAmount { get; set; }
        public DateTime OrderDate { get; set; }
        public AddressDTO ShippingAddress { get; set; } = null!;
        public List<OrderItemDTO> Items { get; set; } = new();
    }
}

```

OrderItemDTO

Create a class file named **OrderItemDTO.cs** within the DTOs folder of ECommerce.Applicaion class library project, then copy and paste the following code. This DTO is used for each item in an order.

```

namespace ECommerce.Application.DTOs
{
    public class OrderItemDTO
    {
        public int OrderId { get; private set; }
        public int ProductId { get; private set; }
        public string? ProductName { get; private set; }
        public decimal UnitPrice { get; private set; }
        public int Quantity { get; private set; }
    }
}

```

AddressDTO

Create a class file named **AddressDTO.cs** within the DTOs folder of ECommerce.Applicaion class library project, then copy and paste the following code. This DTO is used for customer/order addresses.

```
using System.ComponentModel.DataAnnotations;
```

```
namespace ECommerce.Application.DTOs
```

```
{  
    public class AddressDTO  
    {  
        [Required, MaxLength(250)]  
        public string Street { get; set; } = null!;  
  
        [Required, MaxLength(100)]  
        public string City { get; set; } = null!;  
  
        [Required, MaxLength(100)]  
        public string State { get; set; } = null!;  
  
        [Required, MaxLength(20)]  
        public string PostalCode { get; set; } = null!;  
  
        [Required, MaxLength(100)]  
        public string Country { get; set; } = null!;  
    }  
}
```

OrderItemRequestDTO

Create a class file named **OrderItemRequestDTO.cs** within the DTOs folder of ECommerce.Applicaion class library project, then copy and paste the following code. This DTO is used for items in a new order request.

```
using System.ComponentModel.DataAnnotations;
```

```
namespace ECommerce.Application.DTOs
```

```
{  
    public class OrderItemRequestDTO  
    {  
        [Required]  
        public int ProductId { get; set; }  
  
        [Required]  
        [Range(1, int.MaxValue, ErrorMessage = "Quantity must be at least 1")]  
        public int Quantity { get; set; }  
    }  
}
```

CreateOrderRequestDTO

Create a class file named **CreateOrderRequestDTO.cs** within the DTOs folder of ECommerce.Applicaion class library project, then copy and paste the following code. This DTO is used for incoming order creation requests.

```
using System.ComponentModel.DataAnnotations;
```

```
namespace ECommerce.Application.DTOs
```

```
{  
    public class CreateOrderRequestDTO  
    {  
        [Required]  
        public int CustomerId { get; set; }  
  
        [Required]  
        public AddressDTO ShippingAddress { get; set; } = null!;  
    }  
}
```

```

        [Required]
        [MinLength(1, ErrorMessage = "At least one order item is required")]
        public List<OrderItemRequestDTO> Items { get; set; } = new();
    }
}

```

IOrderService Interface

Create a class file named **IOrderService.cs** within the Services folder of ECommerce.Applicaion class library project, then copy and paste the following code. Expose methods to place orders, get order by Id.

```

using ECommerce.Application.DTOs;

namespace ECommerce.Application.Services
{
    public interface IOrderService
    {
        Task<int> PlaceOrderAsync(CreateOrderRequestDTO orderRequest);
        Task<OrderDTO?> GetOrderByIdAsync(int orderId);
    }
}

```

OrderService Implementation

Create a class file named **OrderService.cs** within the Services folder of ECommerce.Applicaion class library project, then copy and paste the following code. Validates order creation (checks customer, stock, etc.), uses domain service for business rules, persists using repositories, maps to DTOs for API.

```

using AutoMapper;
using ECommerce.Application.DTOs;
using ECommerce.Domain.Entities;
using ECommerce.Domain.Repositories;
using ECommerce.Domain.Services;

namespace ECommerce.Application.Services
{
    public class OrderService : IOrderService
    {
        private readonly IProductRepository _productRepository;
        private readonly IOrderRepository _orderRepository;
        private readonly OrderDomainService _orderDomainService;
        private readonly ICustomerRepository _customerRepository;
        private readonly IMapper _mapper;

        public OrderService(
            IProductRepository productRepository,
            IOrderRepository orderRepository,
            ICustomerRepository customerRepository,
            OrderDomainService orderDomainService,
            IMapper mapper)
        {
            _productRepository = productRepository;
            _orderRepository = orderRepository;
            _customerRepository = customerRepository;
            _orderDomainService = orderDomainService;
            _mapper = mapper;
        }

        public async Task<int> PlaceOrderAsync(CreateOrderRequestDTO request)
        {
            var customer = await _customerRepository.GetByIdAsync(request.CustomerId);

```

```

        if (customer == null)
            throw new Exception($"Customer with Id {request.CustomerId} does not exist.");

        var shippingAddress = new Address(request.ShippingAddress.Street,
            request.ShippingAddress.City,
            request.ShippingAddress.State,
            request.ShippingAddress.PostalCode,
            request.ShippingAddress.Country);

        var order = new Order(request.CustomerId, shippingAddress);

        foreach (var item in request.Items)
        {
            var product = await _productRepository.GetByIdAsync(item.ProductId);
            if (product == null)
                throw new Exception($"Product with Id {item.ProductId} not found.");

            order.AddItem(product, item.Quantity);
        }

        // Use the domain service to validate order
        if (!_orderDomainService.CanPlaceOrder(customer, order.Items.ToList()))
            throw new Exception("Order cannot be placed due to domain validation failure.");

        await _orderRepository.AddAsync(order);

        return order.Id; // Return generated Id
    }

    public async Task<OrderDTO?> GetOrderByIdAsync(int orderId)
    {
        var order = await _orderRepository.GetByIdAsync(orderId);
        if (order == null)
            return null;

        return _mapper.Map<OrderDTO>(order);
    }
}

```

Automapper Configuration

Configure AutoMapper profiles in Application Layer. So, create a class file named **MappingProfile.cs** within the Mappings folder of ECommerce.Applicaion class library project, then copy and paste the following code. Configures how domain models map to DTOs and vice versa, making conversions automatic.

```

using AutoMapper;
using ECommerce.Application.DTOs;
using ECommerce.Domain.Entities;

namespace ECommerce.Application.Mappings
{
    public class MappingProfile : Profile
    {
        public MappingProfile()
        {
            // Product mappings
            CreateMap<Product, ProductDTO>();
            CreateMap<CreateProductDTO, Product>();
        }
    }
}

```

```

        // Order mappings
        CreateMap<Order, OrderDTO>();
        CreateMap<OrderItem, OrderItemDTO>();

        // Address mapping
        CreateMap<Address, AddressDTO>();
    }
}
}

```

Presentation Layer — API Controllers

Let us implement the Application Layer where we will create API Controllers to expose the API endpoints which will be consumed by the clients.

ProductsController

Create an Empty API Controller named **ProductsController.cs** within the Controllers folder of ECommerce.API project, then copy and paste the following code. The Products Controller exposes endpoints to get products or create new products via HTTP calls. It communicates only with IProductService.

```

using ECommerce.Application.DTOs;
using ECommerce.Application.Services;
using Microsoft.AspNetCore.Mvc;

namespace ECommerce.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class ProductsController : ControllerBase
    {
        private readonly IProductService _productService;

        public ProductsController(IProductService productService)
        {
            _productService = productService;
        }

        [HttpGet]
        public async Task<ActionResult> GetAllProducts()
        {
            var products = await _productService.GetAllProductsAsync();
            return Ok(products);
        }

        [HttpGet("{id}")]
        public async Task<ActionResult> GetProductById(int id)
        {
            var product = await _productService.GetProductByIdAsync(id);
            if (product == null)
                return NotFound();
            return Ok(product);
        }

        [HttpPost]
        public async Task<ActionResult> CreateProduct(CreateProductDTO productDto)
        {
            var product = await _productService.AddProductAsync(productDto);
            return CreatedAtAction(nameof(GetProductById), new { id = product.Id }, product);
        }
    }
}

```

```
}
```

OrdersController

Create an Empty API Controller named **OrdersController.cs** within the Controllers folder of ECommerce.API project, then copy and paste the following code. The OrdersController exposes endpoints to place new orders and retrieve existing orders via HTTP calls. It communicates only with IOrderService.

```
using Microsoft.AspNetCore.Mvc;
using ECommerce.Application.DTOs;
using ECommerce.Application.Services;

namespace ECommerce.API.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    public class OrdersController : ControllerBase
    {
        private readonly IOrderService _orderService;
        public OrdersController(IOrderService orderService)
        {
            _orderService = orderService;
        }

        [HttpPost]
        public async Task<IActionResult> CreateOrder([FromBody] CreateOrderRequestDTO request)
        {
            try
            {
                int newOrderId = await _orderService.PlaceOrderAsync(request);
                return Ok(new
                {
                    Message = "Order placed successfully",
                    OrderId = newOrderId
                });
            }
            catch (Exception ex)
            {
                return BadRequest(new { Error = ex.Message });
            }
        }

        [HttpGet("{id}")]
        public async Task<IActionResult> GetOrder(int id)
        {
            var order = await _orderService.GetOrderByIdAsync(id);
            if (order == null)
                return NotFound();

            return Ok(order);
        }
    }
}
```

appsettings.json File

This file contains the application settings, including the database connection string. So, please modify the appsettings.json file as follows:

```
{
```

```

"Logging": {
  "LogLevel": {
    "Default": "Information",
    "Microsoft.AspNetCore": "Warning"
  }
},
"AllowedHosts": "*",
"ConnectionStrings": {
  "DefaultConnection": "Server=LAPTOP-6P5NK25R\\SQLSERVER2022DEV;Database=ECommerceDB;Trusted_Connection=True;TrustServerCertificate=True;"
}
}

```

Install Microsoft.EntityFrameworkCore.Design Package in API Project:

EF Core tools use design-time services from the startup project to:

- Read configuration (like connection strings).
- Instantiate the DbContext.
- Generate migrations correctly.

In our example, our connection string is stored in API project and DbContext in Infrastructure project. So, read the connection string from API Project, please execute the following command in Package Manage Console. Please ensure to select the API Project:

Install-Package Microsoft.EntityFrameworkCore.Design

Middleware and Services Configuration:

Register services and Middleware components.

- Registers all services, repositories, DbContext, and AutoMapper.
- Sets up Swagger for API documentation.
- Applies JSON serialization settings.
- Configures dependency injection.

So, please modify the Program class file of ECommerce.API project as follows:

```

using ECommerce.Application.Mappings;
using ECommerce.Application.Services;
using ECommerce.Domain.Repositories;
using ECommerce.Domain.Services;
using ECommerce.Infrastructure.Persistence;
using ECommerce.Infrastructure.Repositories;
using Microsoft.EntityFrameworkCore;

namespace ECommerce.API
{
    public class Program
    {
        public static void Main(string[] args)
        {
            var builder = WebApplication.CreateBuilder(args);

            // Add services to the container.
            builder.Services.AddControllers()
                .AddJsonOptions(options =>
                {
                    //Disable Camel case naming conventions for JSON Serialization and Deserialization
                    options.JsonSerializerOptions.PropertyNamingPolicy = null;
                });

```

```

// Learn more about configuring Swagger/OpenAPI at
https://aka.ms/aspnetcore/swashbuckle
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

builder.Services.AddDbContext<ECommerceDbContext>(options =>
options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));

builder.Services.AddScoped<IProductRepository, ProductRepository>();
builder.Services.AddScoped<IOrderRepository, OrderRepository>();
builder.Services.AddScoped<ICustomerRepository, CustomerRepository>();

builder.Services.AddScoped<IProductService, ProductService>();
builder.Services.AddScoped<IOrderService, OrderService>();
builder.Services.AddScoped<OrderDomainService>();

builder.Services.AddAutoMapper(typeof(MappingProfile));

var app = builder.Build();

// Configure the HTTP request pipeline.
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

app.UseAuthorization();

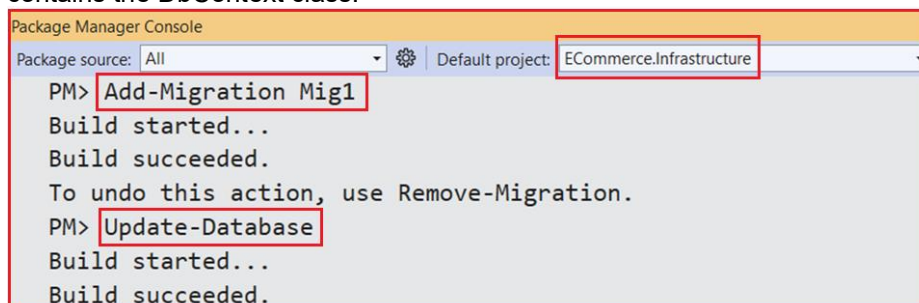
app.MapControllers();

app.Run();
}
}
}

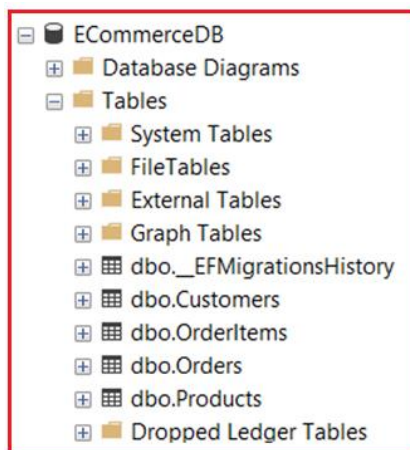
```

Creating and Applying Database Migration:

In Visual Studio, open the Package Manager Console and executes the **Add-Migration** and **Update-Database** command as follows to generate the Migration file and then apply the Migration file to create the database and required tables. Please execute these commands in the Infrastructure project which contains the DbContext class.



Once you execute the above commands, verify the database, you should see the ECommerceDB database with the required tables as shown in the below image.



Why Address Stored as an Owned Entity?

Address is a value object, representing a concept that has no independent identity outside its owning entity (Order). It doesn't make sense to query or update addresses separately.

Keeping the address data in the same table:

- Simplifies the schema (fewer joins).
- Improves query performance for reading order with address.
- Keeps the address snapshot exactly as it was at the time of order, avoiding issues if customer changes address later.
- No need to manage a separate table or relationships for address, reducing complexity for inserts/updates.

Historical Accuracy: Order's shipping address is typically a snapshot, you want to preserve the address at the time of order, even if customer updates their profile later. Storing it embedded avoids accidental updates.

Understanding Key DDD Elements

Entities:

- An entity is an object with a unique and persistent identity that remains constant throughout its lifecycle, even if its attributes change.
- Example: In an e-commerce platform, a Customer is an entity. Each customer has a unique CustomerId; even if their address or phone number changes, they are always identified by that same CustomerId.

Value Objects:

- A value object represents attributes or characteristics of the domain and does not have a unique identity. It is defined entirely by its properties.
- Example: An Address is a value object—it's simply a set of details like street, city, and postal code

Aggregate:

- An aggregate is a group of related entities and value objects that are managed together as one unit, with a single root entity (the aggregate root) as the only way to interact with the group.
- **Example:** An Order aggregate includes:
 1. The Order (aggregate root) with a unique OrderId
 2. A list of OrderItems (each with a ProductId and quantity)
 3. The Shipping Address (a value object)
- All changes, like adding or removing items or updating the shipping address, must go through the Order entity, which enforces rules such as "an order must have at least one item."

Domain Events:

- A domain event is a notification that something important has happened within your domain, allowing other parts of the system to react accordingly.

- Example: When an Order is placed, an OrderPlaced event can be created to trigger actions like sending a confirmation email, updating inventory, or starting the shipping process.

Domain Services:

- A domain service contains business logic that doesn't naturally belong to any single entity or value object, often because it involves coordination between several objects.
- **Example:** An Order Processing Service might validate a customer's information, check that an order has at least one item, and handle payment—all tasks that span multiple entities and can't be handled by just one.

Testing APIs:

Create Order:

```
{
  "CustomerId": 1,
  "ShippingAddress": {
    "Street": "123 Main St",
    "City": "Cityville",
    "State": "Stateville",
    "PostalCode": "12345",
    "Country": "Countryland"
  },
  "Items": [
    { "ProductId": 1, "Quantity": 2 },
    { "ProductId": 3, "Quantity": 1 }
  ]
}
```

Domain-Driven Design (DDD) Architecture provides a powerful approach to building complex business applications by focusing on the domain and its logic as the heart of the system. With clear separation into Domain, Application, Infrastructure, and Presentation layers, DDD enables maintainable, flexible, and testable applications.

By implementing DDD using ASP.NET Core Web API, developers can build robust and scalable systems like E-Commerce platforms that reflect real-world business complexities accurately, while maintaining high code quality and adaptability to changing requirements.